Especificação Técnica e Guia de Implementação: MVP Barbearia (BarberApp)

Versão 2 — revisada para fechar lacunas de escopo antes do início do desenvolvimento assistido.

Este documento estabelece o escopo funcional, arquitetural, estrutural e operacional para o desenvolvimento do Produto Mínimo Viável (MVP) de um sistema de gestão e agendamento para barbearia. Projetado para guiar o fluxo de desenvolvimento assistido no Antigravity, este manual aborda desde o esquema relacional até os padrões de integração e roteiro de execução em sprints.

1. Visão Geral e Objetivos do MVP

O objetivo primordial deste MVP é validar o fluxo completo de autoatendimento para barbearias: o cliente visualiza serviços e profissionais disponíveis, seleciona horários dinamicamente sem sobreposição de agenda e confirma a reserva. O back-end em Java/Spring Boot garante integridade relacional e validação concorrente, enquanto o front-end em React oferece experiência ergonômica, inspirada em microaplicativos móveis (mobile-first).

Perfil	Ações Principais no MVP	Critério de Sucesso
Cliente	Consultar catálogo, filtrar profissional, agendar horário vago, visualizar reservas.	Concluir agendamento em menos de 4 cliques sem duplicidade.
Barbeiro	Visualizar agenda cronológica diária, alterar status de atendimento.	Clareza total sobre o próximo cliente e tempo de duração.
Administrador	Cadastrar barbeiros, cadastrar serviços (preço/duração), gerenciar cancelamentos.	Manutenção autônoma do cardápio e profissionais.

> **NOTA DA REVISÃO:** *As ações do Administrador e do Barbeiro descritas acima agora têm endpoints correspondentes nas Seções 6 e 8 — na v1 do documento essas ações não tinham rota associada.*

2. Stack Tecnológica e Arquitetura de Software

A aplicação opera sob o modelo cliente-servidor desacoplado via API REST, com comunicação estritamente tipada e contratos JSON padronizados (camelCase, em português).

- Back-end: Java 17/21 com Spring Boot 3 (Spring Web, Spring Data JPA, Bean Validation, Spring Security — apenas HTTP Basic Auth, PostgreSQL Driver).
- Banco de Dados: PostgreSQL gerenciado com chaves estrangeiras e índices de data.
- Front-end: React 18+ com Vite, TypeScript, Tailwind CSS e Lucide Icons (design com bottom-navigation e cards).
- Comunicação: Axios com instâncias configuradas para baseURL e interceptors.
- Ambiente de Desenvolvimento: Antigravity / IntelliJ IDEA com Git e Maven.

> **NOTA DA REVISÃO:** *Adicionado Spring Security à stack — necessário para proteger os endpoints de administração (Seção 8). Uso restrito a HTTP Basic Auth com um único usuário fixo; sem JWT, sem OAuth2, sem tabela de sessões.*

3. Modelagem de Dados e Esquema Relacional

A integridade referencial é estruturada em torno de quatro tabelas fundamentais. O agendamento é a entidade pivô que une cliente, profissional e serviço. O esquema não foi alterado nesta revisão — os ajustes do MVP são tratados via configuração de aplicação (Seção 4), não via novas tabelas.

Tabela	Campo	Tipo SQL	Restrições / Chaves
usuarios	id, nome, email, telefone, perfil	BIGSERIAL, VARCHAR(100),	PK (id), UNIQUE (email), perfil IN ('CLIENTE','BARBEIRO','ADMIN')

Tabela	Campo	Tipo SQL	Restrições / Chaves
		VARCHAR(100), VARCHAR(20), VARCHAR(20)	
barbeiros	id, usuario_id, especialidade, ativo	BIGSERIAL, BIGINT, VARCHAR(100), BOOLEAN	PK (id), FK (usuario_id REFERENCES usuarios)
servicos	id, nome, descricao, preco, duracao_minutos	BIGSERIAL, VARCHAR(100), TEXT, NUMERIC(10,2), INTEGER	PK (id), duracao_minutos > 0, preco >= 0
agendamentos	id, cliente_id, barbeiro_id, servico_id, data_hora_inicio, data_hora_fim, status	BIGSERIAL, BIGINT, BIGINT, BIGINT, TIMESTAMP, TIMESTAMP, VARCHAR(20)	PK (id), FKs (usuarios, barbeiros, servicos), status IN ('AGENDADO','CONCLUIDO','CANCELADO')

Script DDL Inicial (PostgreSQL):

```
CREATE TABLE usuarios (
    id BIGSERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    telefone VARCHAR(20) NOT NULL,
    perfil VARCHAR(20) NOT NULL DEFAULT 'CLIENTE'
);

CREATE TABLE barbeiros (
    id BIGSERIAL PRIMARY KEY,
    usuario_id BIGINT NOT NULL REFERENCES usuarios(id) ON DELETE CASCADE,
    especialidade VARCHAR(100) NOT NULL,
    ativo BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE servicos (
    id BIGSERIAL PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    descricao TEXT,
    preco NUMERIC(10,2) NOT NULL,
    duracao_minutos INTEGER NOT NULL
);

CREATE TABLE agendamentos (
    id BIGSERIAL PRIMARY KEY,
    cliente_id BIGINT NOT NULL REFERENCES usuarios(id),
    barbeiro_id BIGINT NOT NULL REFERENCES barbeiros(id),
    servico_id BIGINT NOT NULL REFERENCES servicos(id),
    data_hora_inicio TIMESTAMP NOT NULL,
    data_hora_fim TIMESTAMP NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'AGENDADO'
);

CREATE INDEX idx_agendamentos_barbeiro_data ON agendamentos(barbeiro_id, data_hora_inicio);
```

4. Configuração de Expediente e Geração de Slots (NOVO)

Lacuna crítica da v1: sem um horário de funcionamento definido, é impossível calcular a lista de horários livres exigida pelo endpoint de disponibilidade (Seção 6). Esses valores são configuração de aplicação, não uma tabela nova — não há necessidade de CRUD de expediente no MVP.

application.properties:

```
horario.abertura=09:00
horario.fechamento=19:00
slot.duracao.minutos=30
dias.atendimento=SEG,TER,QUA,QUI,SEX,SAB
```

Lido via uma classe anotada com @ConfigurationProperties(prefix = "horario") (mais um pequeno bean para slot.duracao.minutos e dias.atendimento). O algoritmo de geração de slots percorre o intervalo [abertura, fechamento) em passos de slot.duracao.minutos, descarta os slots cuja janela não caiba a duracao_minutos do serviço selecionado, e remove os que colidem com agendamentos existentes (Seção 5).

5. Regra de Negócio Crucial: Conflito de Horários e Geração de Slots

O algoritmo de agendamento deve garantir que não ocorra colisão de intervalos para um mesmo barbeiro. Considera-se conflito se o intervalo solicitado [Início, Fim) interceptar qualquer agendamento existente com status AGENDADO.

Condição de Colisão (SQL/JPA):

```
@Query("""
    SELECT COUNT(a) > 0 FROM Agendamento a
    WHERE a.barbeiro.id = :barbeiroId
      AND a.status = 'AGENDADO'
      AND (:novoInicio < a.dataHoraFim AND :novoFim > a.dataHoraInicio)
""")
boolean existsConflitoHorario(
    @Param("barbeiroId") Long barbeiroId,
    @Param("novoInicio") LocalDateTime novoInicio,
    @Param("novoFim") LocalDateTime novoFim
);
```

NOTA DA REVISÃO: *Correção de concorrência: a v1 do documento descrevia apenas este SELECT, o que não impede que duas requisições simultâneas leiam "sem conflito" antes de qualquer uma commitar. O AgendamentoService deve executar, dentro do MESMO método @Transactional: (1) SELECT ... FOR UPDATE na linha do barbeiro (lock pessimista), (2) a checagem de conflito acima, (3) o INSERT do agendamento. Isso serializa as tentativas concorrentes para aquele barbeiro específico sem exigir nenhuma infraestrutura nova.*

6. Especificação dos Endpoints da API REST

Contratos de comunicação HTTP claros e objetivos para simplificar o consumo do front-end. Lista fechada — endpoints adicionados nesta revisão estão marcados como (NOVO).

Método	Endpoint	Descrição	Retorno
GET	/api/servicos	Lista serviços com preços e tempos de duração.	200 OK
GET	/api/barbeiros	Lista barbeiros ativos e especialidades.	200 OK
POST	/api/barbeiros (NOVO)	Cadastra barbeiro. Protegido por Basic Auth (Admin).	201 Created / 401

Método	Endpoint	Descrição	Retorno
POST	/api/servicos (NOVO)	Cadastra serviço. Protegido por Basic Auth (Admin).	201 Created / 401
POST	/api/usuarios (NOVO)	Cadastra cliente (nome, email, telefone) e retorna o id usado nos agendamentos.	201 Created
GET	/api/agendamentos/disponibilidade?barbeiroId=1&servicoId=3&data=2026-09-15	Retorna horários livres do dia, já considerando a duração do serviço (servicoId incluído nesta revisão).	200 OK
POST	/api/agendamentos	Cria agendamento após validação de concorrência (lock + checagem, Seção 5).	201 Created / 409 Conflict
GET	/api/agendamentos/cliente/{clienteId}	Lista histórico e próximos agendamentos do cliente.	200 OK
GET	/api/agendamentos/barbeiro/{barbeiroId}?data=2026-09-15 (NOVO)	Lista a agenda cronológica diária do barbeiro.	200 OK
PATCH	/api/agendamentos/{id}/cancelar	Cancela uma reserva, liberando o horário.	200 OK / 404 Not Found
PATCH	/api/agendamentos/{id}/concluir (NOVO)	Marca o agendamento como concluído.	200 OK / 404 Not Found

> **NOTA DA REVISÃO:** *Correções: (1) o endpoint de disponibilidade não recebia servicoId na v1, tornando impossível calcular corretamente a duração dos slots; (2) não existiam rotas de escrita para barbeiros/serviços nem rota de agenda do barbeiro, apesar de serem ações descritas na Seção 1; (3) não existia forma de criar um clienteId antes do primeiro agendamento; (4) o status CONCLUIDO existia no enum mas nenhuma rota o utilizava.*

Exemplo de Payload de Criação (POST /api/agendamentos):

```
{
  "clienteId": 1,
  "barbeiroId": 2,
```

```
  "servicoId": 3,
  "dataHoraInicio": "2026-09-15T14:00:00"
}
```

7. Contrato de Erros da API (NOVO)

Todo retorno 4xx/409 da API deve usar exatamente o mesmo formato, implementado uma única vez em um @ControllerAdvice global. Sem isso, cada controller tende a inventar um shape diferente e o front-end precisa tratar múltiplos formatos de erro.

```
{
  "mensagem": "Horário indisponível para este barbeiro.",
  "codigo": "CONFLITO_HORARIO",
  "timestamp": "2026-09-15T14:00:00Z"
}
```

8. Política de Acesso e Autenticação (NOVO)

Lacuna da v1: nenhuma estratégia de autenticação era definida, mas os novos endpoints de administração (Seção 6) precisam de alguma proteção. Para o MVP, a solução mais simples é suficiente — nada de JWT, OAuth2 ou tabela de sessões.

- Mecanismo: Spring Security com HTTP Basic Auth.
- Credenciais: um único usuário fixo, lido de application.properties (admin.user, admin.pass).
- Rotas protegidas: apenas POST /api/barbeiros e POST /api/servicos.
- Todas as demais rotas permanecem públicas no MVP.

9. Fluxo de Cadastro de Cliente (NOVO)

A v1 do documento exigia um clienteId no payload de criação de agendamento (Seção 6), mas não definia como esse id era gerado — não havia nenhum endpoint de cadastro de usuário. O fluxo correto é:

- 1. No primeiro acesso, o front-end coleta nome, email e telefone do cliente.
- 2. Front-end chama POST /api/usuarios, que retorna { id }.
- 3. O id retornado é usado como clienteId em todos os agendamentos subsequentes daquela sessão (armazenado em estado local do React, sem necessidade de login completo no MVP).

10. Estrutura de Diretórios Recomendada

Padronização de diretórios tanto para o ecossistema Spring Boot quanto para o front-end React com Vite.

Back-end (Java / Spring Boot):

```
barberapp-backend/
├── pom.xml
└── src/main/java/com/barberapp/
    ├── config/             # SecurityConfig (Basic Auth), HorarioExpedienteProperties
    ├── controller/         # AgendamentoController, ServicoController, BarbeiroController,
UsuarioController
    ├── dto/                # Request e Response records (DTOs desacoplados)
    ├── model/              # Entidades JPA (@Entity): Usuario, Barbeiro, Servico,
Agendamento
    ├── repository/         # Interfaces Spring Data JPA
    ├── service/             # Regras de negócio e validação de conflito de agenda
    └── exception/          # ErrorResponse + @ControllerAdvice global
```

Front-end (React / Vite):

```
barberapp-frontend/
├── package.json
├── tailwind.config.js
```

```
└── src/
    ├── components/
    │   ├── BottomNav.tsx        # Navegação inferior estilo mobile
    │   ├── ServiceCard.tsx      # Card selecionável de serviço
    │   ├── BarberCard.tsx       # Avatar e detalhes do barbeiro
    │   └── TimeSlotPicker.tsx   # Grade de botões de horários disponíveis
    ├── pages/
    │   ├── Home.tsx             # Resumo e botão rápido de agendar
    │   ├── CadastroCliente.tsx  # Coleta de dados no primeiro acesso (POST /api/usuarios)
    │   ├── AgendamentoFlow.tsx  # Fluxo wizard (Serviço -> Barbeiro -> Data/Hora)
    │   └── MeusAgendamentos.tsx # Lista com status e ação de cancelamento
    └── services/
        └── api.ts               # Conexão com Axios apontando para o Spring Boot
```

11. Roteiro Passo a Passo para Desenvolvimento no Antigravity

Siga esta sequência lógica de prompts e commits no Antigravity para manter a clareza didática e evolução controlada:

- Etapa 1 - Bootstrap & Configuração: Criar o projeto Spring Boot, configurar application.properties (conexão PostgreSQL + constantes de expediente da Seção 4).
- Etapa 2 - Entidades e Repositórios: Implementar Usuario, Barbeiro, Servico e Agendamento com anotações JPA. Criar as interfaces JpaRepository equivalentes.
- Etapa 3 - Service de Conflito de Horário: Implementar AgendamentoService com cálculo de dataHoraFim, geração de slots (Seção 4) e checagem de disponibilidade com lock pessimista (Seção 5).
- Etapa 4 - Camada Web (Controllers & DTOs): Criar todos os endpoints da Seção 6, incluindo os de administração, com DTOs (records) e o @ControllerAdvice de erro (Seção 7).
- Etapa 5 - Segurança: Configurar Spring Security com HTTP Basic Auth apenas nas rotas de administração (Seção 8).
- Etapa 6 - Front-end Mobile-First: Inicializar o projeto React com Tailwind. Construir cards de serviços, barra inferior fixa e a tela de cadastro de cliente (Seção 9).
- Etapa 7 - Integração e Teste Ponta a Ponta: Conectar via Axios, efetuar agendamentos reais e testar a barreira de agendamento duplicado com requisições concorrentes.
- Etapa 8 - Seed de Dados: Rodar o script da Seção 12 para popular o ambiente com dados de teste.

12. Plano de Dados de Teste / Seed (NOVO)

Sem dados de teste, os critérios de aceite (Seção 13) não podem ser validados manualmente sem cadastro repetitivo via UI. Rodar este script após a criação do schema:

```
INSERT INTO usuarios (nome, email, telefone, perfil) VALUES
  ('Admin Barbearia', 'admin@barberapp.com', '11999990000', 'ADMIN'),
  ('João Barbeiro', 'joao@barberapp.com', '11999990001', 'BARBEIRO'),
  ('Marcos Barbeiro', 'marcos@barberapp.com', '11999990002', 'BARBEIRO');

INSERT INTO barbeiros (usuario_id, especialidade, ativo) VALUES
  (2, 'Corte e Barba', true),
  (3, 'Degradê e Coloração', true);

INSERT INTO servicos (nome, descricao, preco, duracao_minutos) VALUES
  ('Corte Simples', 'Corte na tesoura ou máquina', 40.00, 30),
  ('Corte + Barba', 'Corte completo com barba desenhada', 65.00, 60),
  ('Coloração', 'Coloração completa', 90.00, 90);
```

13. Critérios de Aceite para Validação do MVP

- A listagem de horários desabilita botões de horários já ocupados pelo barbeiro selecionado, considerando a duração do serviço escolhido (servicoId).

- O back-end rejeita com status 409 qualquer tentativa de inserir um horário colidente, inclusive sob duas requisições concorrentes para o mesmo slot (teste com 2 chamadas em paralelo).
- POST /api/barbeiros e POST /api/servicos sem credenciais de Basic Auth retornam 401.
- O cliente consegue se cadastrar via POST /api/usuarios e usar o id retornado para agendar, sem passos manuais no banco.
- O barbeiro consegue visualizar sua agenda diária via GET /api/agendamentos/barbeiro/{id}.
- O layout na resolução 375px a 430px (telas móveis comuns) apresenta navegação limpa, sem barra de rolagem horizontal.
- O cliente consegue cancelar seu agendamento, liberando imediatamente o slot na próxima busca de disponibilidade.